

DOCUMENTO DE DISEÑO UML

Fase 1 — Proyecto Integrador: Diseño de Software

Sistema de Préstamo de Equipos Tecnológicos

Dominio D3 — Sistema de préstamo de recursos

Estudiante	roddyt
Docente	Victoria García Velásquez, MGP.
Modalidad	Individual (registrado bajo el dominio D3)
Entrega	Fase 1 — Fin de semana 7
Fecha	24 de julio de 2026

1. Descripción del sistema — Enfoque 4+1

El sistema modela el **préstamo de equipos tecnológicos** (laptops, proyectores, cámaras y similares) dentro de una institución educativa u oficina. Un único rol operativo, el **Encargado**, registra los préstamos y devoluciones de equipos a nombre de los usuarios (estudiantes o docentes), consulta el historial de préstamos activos y recibe alertas cuando un préstamo se atrasa.

El alcance se mantiene deliberadamente acotado, tal como fue indicado por la docente para esta entrega:

- **Incluido:** registro de préstamo, registro de devolución (con posibilidad de marcar el equipo como dañado), y consulta de historial/atrasos.
- **No incluido (fuera de alcance):** autenticación multiusuario, cobro de multas monetarias, reservas anticipadas por calendario, y envío real de correos/SMS (se deja la interfaz preparada para ello, ver sección 5).

A continuación se describe el sistema bajo las cinco vistas del modelo 4+1 de Kruchten. Las vistas lógica, de procesos, de desarrollo y física se apoyan y validan mutuamente a través de la vista de escenarios (los casos de uso), que es la que las conecta a todas.

1.1 Vista de Escenarios (Casos de uso)

Es la vista que amarra a las demás: cada decisión de clases, secuencia de mensajes y organización de módulos existe para soportar los 3 casos de uso principales del sistema (Registrar Préstamo, Registrar Devolución y Consultar Historial de Préstamos). El detalle completo está en la **sección 2**.

1.2 Vista Lógica

Se compone de las clases de dominio (Usuario, Equipo, Prestamo), los servicios de coordinación (GestorPrestamos), los repositorios que abstraen el almacenamiento (UsuarioRepository, EquipoRepository, PrestamoRepository) y la interfaz de notificación (NotificadorAtraso) que implementa el patrón Observer. El detalle completo está en la **sección 3**.

1.3 Vista de Procesos

Dado el alcance mínimo del proyecto, el sistema corre como **un solo proceso, de un solo hilo**: el Encargado interactúa secuencialmente con `GestorPrestamos`, que a su vez invoca a los repositorios de forma síncrona. No existe concurrencia real entre múltiples usuarios operando al mismo tiempo, por lo que no se requieren mecanismos de bloqueo o control transaccional en esta fase. Si el sistema creciera a un escenario con múltiples encargados operando simultáneamente, el punto de extensión sería un control optimista sobre el atributo `estado` de `Equipo` antes de confirmar un préstamo.

1.4 Vista de Desarrollo

La organización de módulos del código base (Fase 2) refleja directamente la vista lógica, separando responsabilidades para facilitar las refactorizaciones de la Fase 3:

- `app/Models/` — entidades de dominio (`Usuario`, `Equipo`, `Prestamo`) como modelos Eloquent, y sus enumeraciones de estado en `app/Enums/`.
- `app/Repositories/` — abstraen el acceso a cada entidad detrás de una interfaz simple (`buscar/guardar/listar`), aunque internamente usen Eloquent.
- `app/Services/` — `GestorPrestamos`, que orquesta los 3 casos de uso.
- `app/Notifiers/` — interfaz `NotificadorAtraso` y su implementación `ConsolaNotificador`.
- `tests/Feature/` — pruebas unitarias (PHPUnit) de cada caso de uso y del patrón Observer.

La dependencia siempre apunta hacia adentro: `Services` depende de los contratos de `Repositories` y `Notifiers`, nunca al revés.

1.5 Vista Física

El despliegue es intencionalmente simple: **PHP 8.3 con Laravel, corriendo en un contenedor Docker**, con **SQLite** (un único archivo local) como persistencia — sin servidor de base de datos independiente, sin componentes distribuidos, sin dependencias de red. Usar Docker evita instalar PHP, Composer o Laravel directamente en la máquina de desarrollo; usar SQLite evita levantar un motor de base de datos aparte. Es la misma decisión de fondo que en las demás vistas: mantener el sistema sencillo, tal como pidió la docente, sin sacrificar la coherencia del diseño orientado a objetos.

2. Diagrama de Casos de Uso

Actor único: **Encargado**. Se identifican 3 casos de uso principales, cada uno con una relación «include» o «extend» hacia un caso de uso secundario:

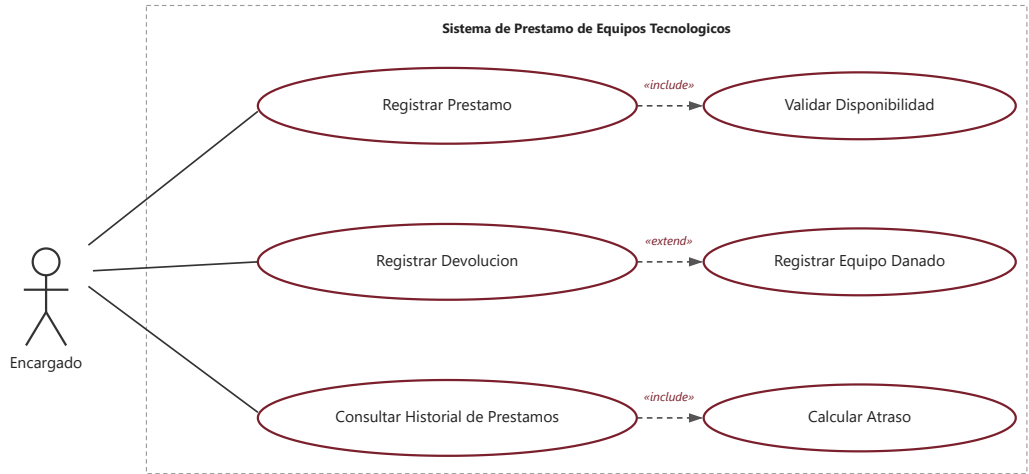


Figura 1. Diagrama de casos de uso del sistema de préstamo de equipos.

Caso de uso	Relación	Descripción breve
UC1 — Registrar Préstamo	«include» Validar Disponibilidad	El Encargado selecciona un usuario y un equipo; el sistema valida que el usuario esté habilitado y el equipo disponible antes de crear el préstamo.
UC2 — Registrar Devolución	«extend» Registrar Equipo Dañado	El Encargado cierra un préstamo activo. Si el equipo regresa en mal estado, se activa el flujo de extensión que cambia el estado del equipo a "dañado" en vez de "disponible".
UC3 — Consultar Historial de Préstamos	«include» Calcular Atraso	El Encargado lista los préstamos activos; por cada uno se incluye el cálculo de si está atrasado respecto a la fecha de devolución esperada.

Por qué «include» y no «extend» (y viceversa): "Validar Disponibilidad" y "Calcular Atraso" son pasos que *siempre* ocurren dentro de su caso de uso base, por eso son «include». "Registrar Equipo Dañado" es opcional — solo ocurre si el encargado marca el equipo como dañado al momento de la devolución — por eso es «extend» con un punto de extensión condicional.

3. Diagrama de Clases

El diseño usa 8 clases (dentro del rango de 6 a 10 exigido), con responsabilidades separadas entre entidades de dominio, repositorios y el servicio de coordinación:

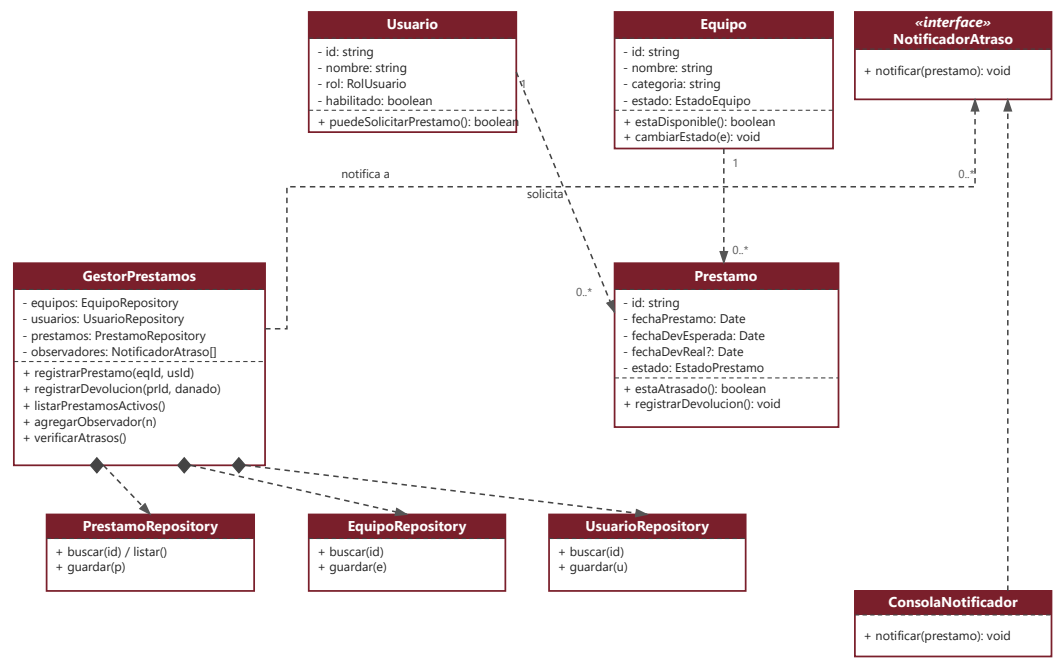


Figura 2. Diagrama de clases con relaciones y multiplicidades.

Clase	Responsabilidad principal
Usuario	Representa a quien solicita equipos; sabe si está habilitado para pedir un préstamo.
Equipo	Representa un recurso prestable; conoce su propio estado (disponible, prestado, dañado).
Prestamo	Representa la transacción de préstamo; calcula si está atrasado y gestiona su propio cierre.
GestorPrestamos	Orquesta los 3 casos de uso; es el único punto de entrada a la lógica de negocio.
UsuarioRepository / EquipoRepository / PrestamoRepository	Abstraen el acceso a cada entidad (Repository pattern) por encima de Eloquent, permitiendo cambiar el motor de base de datos sin tocar GestorPrestamos.
NotificadorAtraso (interfaz) / ConsolaNotificador	Desacoplan la notificación de atrasos del resto del sistema (patrón Observer, justificado en la sección 5).

(implementación)	
------------------	--

Multiplicidades clave: un `Usuario` puede tener 0..* préstamos a lo largo del tiempo, y cada `Prestamo` pertenece a exactamente 1 usuario; análogamente, un `Equipo` puede tener 0..* préstamos históricos, pero solo puede estar en 1 préstamo activo a la vez (regla de negocio validada en tiempo de ejecución, no expresable directamente como multiplicidad estática).

4. Diagramas de Secuencia (casos de uso críticos)

Se modelan los dos casos de uso más críticos del sistema: **Registrar Préstamo** (porque concentra las validaciones de negocio) y **Registrar Devolución** (porque dispara tanto la extensión de equipo dañado como la notificación de atraso vía Observer).

4.1 Registrar Préstamo

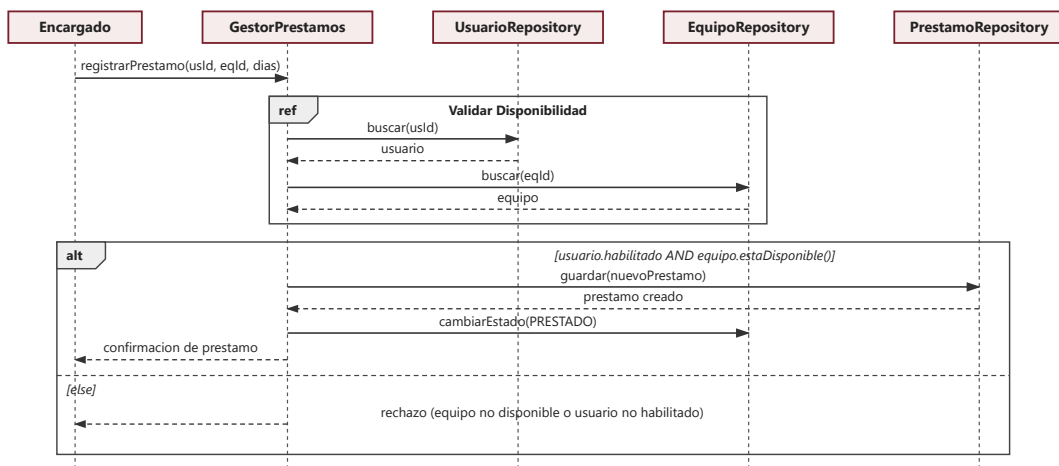


Figura 3. Secuencia de Registrar Préstamo.

4.2 Registrar Devolución

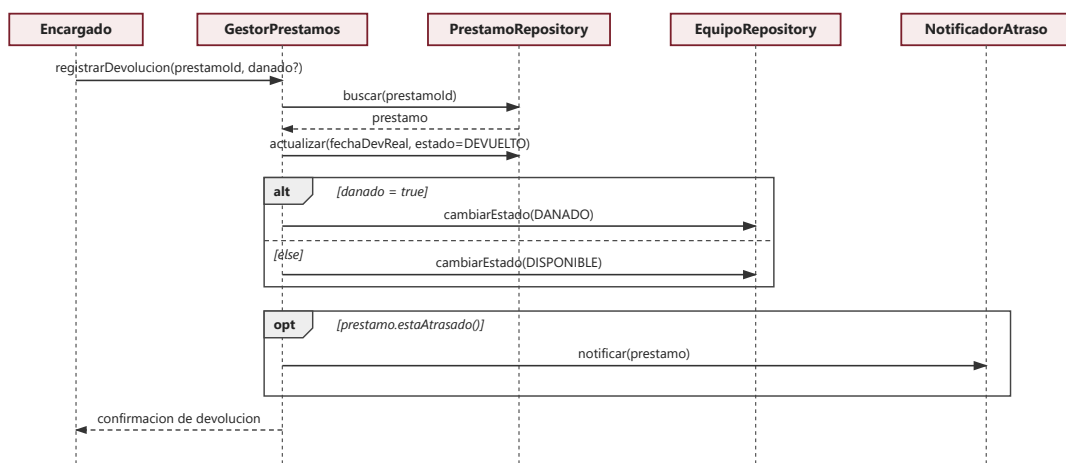


Figura 4. Secuencia de Registrar Devolución.

5. Justificación del patrón de diseño aplicado: Observer

Problema: cuando un préstamo pasa a estado atrasado, alguien debe ser notificado. Hoy esa notificación es un simple mensaje por consola, pero es razonable esperar que en una siguiente iteración se quiera notificar también por correo electrónico, SMS, o un panel web. Si `GestorPrestamos` conociera directamente los detalles de "cómo notificar", cada canal nuevo obligaría a modificar su código — violando el Principio de Abierto/Cerrado (OCP).

Solución aplicada: se define la interfaz `NotificadorAtraso` con el método `notificar(prestamo)`. `GestorPrestamos` mantiene una lista de observadores (`agregarObservador()`) y, cuando detecta un préstamo atrasado (`verificarAtrasos()` o durante `registrarDevolucion()`), simplemente recorre la lista y llama a `notificar()` en cada uno — sin saber qué hace cada implementación concretamente. La única implementación de esta fase es `ConsolaNotificador`, pero agregar `EmailNotificador` en el futuro no requeriría tocar una sola línea de `GestorPrestamos`.

Dónde se ve en los diagramas: la relación de realización entre `ConsolaNotificador` y «interface» `NotificadorAtraso` en la Figura 2, y la llamada `notificar(prestamo)` dentro del bloque `alt` de la Figura 4.

Por qué Observer y no otro patrón: se consideró *Strategy* para el cálculo de atraso, pero ese cálculo es una sola regla fija (`hoy > fechaDevolucionEsperada`), no una familia de algoritmos intercambiables — por lo que *Strategy* habría sido sobreingeniería para el alcance de este proyecto. *Observer*, en cambio, responde a una necesidad real y concreta: múltiples posibles "suscriptores" a un mismo evento (préstamo atrasado), que es exactamente el problema que este patrón resuelve.